P.Vishwa Nandan(192473015)

PART A – PERSISTENCE DESIGN & PSEUDOCODE

TASK A1 – DESIGN A FILE FORMAT (7 MARKS)

1. Aim

To design a suitable file-storage format for university bus-pass records using delimited text files and binary files. The design must support adding new passes, searching by pass ID, listing passes by route number, and handling damaged or incomplete records.

2. Problem Description

The university transport office issues approximately 8,000 bus passes every year.

Each bus-pass record contains the following information:

1. Pass ID
2. Student Register Number
3. Student Name
4. Route Number
5. Validity Date
6. Amount Paid

The transport office requires a file-based system that supports three operations:

- Add a new bus pass.
- Find a bus pass using its Pass ID.
- List all bus passes belonging to a particular route.

The file format should be easy to maintain and should handle damaged records safely.

3. Task A1.1 – Represent the Record (2 Marks)

Two file-storage formats are considered:

1. Delimited text file.
2. Binary file using DataOutputStream.

3.1 (a) Delimited Text File Format

A delimited text file stores records as readable text. Each field is separated using a delimiter.

Selected delimiter: Comma (,)

The file uses CSV (Comma-Separated Values) format.

Record Layout

Field Order	Field Name	Data Type	Example
1	Pass ID	String	P1001
2	Register Number	String	192311001
3	Name	String	Arun Kumar
4	Route Number	Integer	12
5	Validity Date	Date	2027-03-31
6	Amount Paid	Decimal	4500.00
7	CRC32 Checksum	Integer	Integrity value

The checksum is an additional technical field used to detect accidental record corruption.

Example Text File

P1001,192311001,Arun Kumar,12,2027-03-31,4500.00,checksum1

P1002,192311002,"Divya, R",7,2027-06-30,3800.00,checksum2

P1003,192311003,Ravi Kumar,15,2027-12-31,4200.00,checksum3

The checksum values shown above are illustrative placeholders.

How Is a Name Containing the Delimiter Handled?

A name may contain a comma, which is also the selected delimiter.

For example:

Divya, R

If the value is stored without protection, the program may incorrectly treat it as two separate fields.

To prevent this, CSV quoting rules are used.

The field is enclosed within double quotes:

"Divya, R"

If a name contains a double quote, the quote is escaped by writing two double quotes.

Example:

"Arun ""AK"" Kumar"

Newline characters inside fields should be escaped or rejected so that one physical line always represents one complete record.

Advantages of Delimited Text Files

1. Easy to read and edit.
2. Can be opened using text editors and spreadsheet applications.
3. Simple to transfer between systems.
4. Suitable for backup and reporting.
5. Easy to implement using BufferedReader and BufferedWriter.

Disadvantages

1. Requires parsing and conversion of values.
2. Searching requires scanning the file without an index.
3. Uses more storage than compact binary representations in many cases.
4. Special characters must be escaped correctly.

3.2 (b) Binary File Using DataOutputStream

A binary file stores data in a machine-readable format instead of directly readable text.

Java provides DataOutputStream to write primitive data types and DataInputStream to read them.

Binary Record Layout

The following field order is selected.

Field Order	Field Name	DataOutputStream Method	Stored Data
1	File Magic Number	writeInt()	Format identification
2	Format Version	writeInt()	Version number
3	Pass ID	writeUTF()	String
4	Register Number	writeUTF()	String
5	Student Name	writeUTF()	String
6	Route Number	writeInt()	Integer
7	Validity Date	writeLong()	Date as epoch day
8	Amount Paid	writeDouble()	Double

9	CRC32 Checksum	writeLong()	Integrity value

The magic number identifies the file format, and the version number helps identify changes to the record structure.

Example Binary File Structure

```
+---------------------------+
| Magic Number (4 bytes)    |
+---------------------------+
| Format Version (4 bytes)  |
+---------------------------+
| Pass ID (writeUTF)        |
+---------------------------+
| Register Number (writeUTF) |
+---------------------------+
| Student Name (writeUTF)   |
+---------------------------+
| Route Number (4 bytes)    |
+---------------------------+
| Validity Date (8 bytes)   |
+---------------------------+
| Amount Paid (8 bytes)     |
+---------------------------+
| CRC32 Checksum (8 bytes)  |
+---------------------------+
```

The string fields written using writeUTF() contain length information, so their sizes can vary.

The validity date is stored as the number of days since 1 January 1970 using Java's LocalDate epoch-day representation.

Example Binary Writing Operations

```
DataOutputStream out =
    new DataOutputStream(
```

```
        new FileOutputStream("buspasses.dat", true)
    );

out.writeUTF("P1001");

out.writeUTF("192311001");

out.writeUTF("Arun Kumar");

out.writeInt(12);

out.writeLong(
    LocalDate.of(2027, 3, 31).toEpochDay()
);

out.writeDouble(4500.00);

out.close();
```

This demonstrates writing one record's business fields. A complete file must also include the file header and integrity information described above.

Advantages of Binary Files

1. Stores numeric values directly in binary representation.
2. Avoids repeated conversion from text to numbers.
3. Can be more compact than text-based storage.
4. Suitable for applications that process structured data.

Disadvantages

1. Cannot be read easily using a normal text editor.
2. Requires a matching reading and writing format.
3. Changes to the record structure require version management.

4. Corrupted binary records can be harder to recover.

3.3 Comparison of Both File Formats

Feature	Delimited Text File	Binary File
Readability	Human-readable	Not human-readable
Storage	Text representation	Binary representation
Numeric data	Requires conversion	Stored using primitive types
Searching	Line-by-line scanning	Record-by-record reading
Compatibility	Easy to exchange	Requires compatible format
Special character handling	Quoting and escaping	Length-prefixed strings
Maintenance	Easier	Requires format versioning
Suitable for	Simple record storage and reporting	Structured application storage

Design Decision

For the university transport office, the delimited text format is selected for the initial implementation because 8,000 records per year is manageable, and the records are easy to inspect, maintain, and export.

The binary format is an alternative when compact storage and structured binary processing are more important.

4. Task A1.2 – Design the Operations (3 Marks)

Three main operations are required:

1. Add a new bus pass.
2. Find a bus pass using its Pass ID.
3. List all passes belonging to a particular route.

The text-file design uses character streams and buffered streams.

4.1 Stream Classes Used

FileReader

Reads character data from a text file.

It is suitable for reading CSV records.

BufferedReader

Wraps FileReader and reads data efficiently using an internal buffer.

Its readLine() method reads one complete record at a time.

FileWriter

Writes character data into a text file.

It supports append mode for adding new records.

BufferedWriter

Wraps FileWriter and improves writing efficiency.

Its newLine() method writes the record separator.

FileInputStream and FileOutputStream

These are byte streams.

They are suitable for reading and writing binary files, such as records written using DataOutputStream and DataInputStream.

4.2 Operation 1 – Add a New Bus Pass

Aim: To add a new bus-pass record at the end of the text file.

Pseudocode

ALGORITHM: ADD_BUS_PASS

INPUT: Pass ID, Register Number, Name, Route Number, Validity Date, Amount Paid.

OUTPUT: Confirmation of successful insertion or an error message.

BEGIN

1. READ Pass ID, Register Number, Name,
   Route Number, Validity Date and Amount Paid.

2. VALIDATE all input fields.

3. IF any field is invalid THEN
   DISPLAY "Invalid bus-pass information".
   RETURN.

END IF.

4. CHECK whether the Pass ID already exists
by calling FIND_BUS_PASS.

5. IF Pass ID already exists THEN
DISPLAY "Pass ID already exists".
RETURN.
END IF.

6. CONVERT the record into CSV format.

7. ESCAPE special characters in text fields
according to CSV rules.

8. CALCULATE the checksum for the record.

9. OPEN the text file using:
FileWriter in append mode.
BufferedWriter for efficient writing.

10. WRITE the complete record.

11. WRITE a newline character.

12. FLUSH the buffered writer.

13. CLOSE the file.

14. DISPLAY "Bus pass added successfully".

END.

Explanation

1. The user enters the six bus-pass details.
2. The application validates the entered values.
3. It checks whether the Pass ID already exists.
4. The record is converted into CSV format.
5. Text fields containing commas are enclosed in double quotes.
6. A checksum is calculated to help detect damaged records.
7. BufferedWriter appends the record to the existing file.
8. The file is closed after writing.

Stream Justification

FileWriter is used in append mode so that existing records are not deleted.

BufferedWriter reduces repeated writing operations and improves efficiency.

4.3 Operation 2 – Find a Bus Pass by Pass ID

Aim: To search the file and return the record matching the given Pass ID.

Pseudocode

ALGORITHM: FIND_BUS_PASS

INPUT: Required Pass ID.

OUTPUT: Matching bus-pass record or "Pass not found".

BEGIN

1. READ the required Pass ID.

2. IF the text file does not exist THEN
   DISPLAY "Bus-pass file not found".
   RETURN.
  END IF.

3. OPEN the file using:

    FileReader.

    BufferedReader.

4. SET LineNumber = 0.

5. WHILE file has another line DO

    READ the next line.

    INCREMENT LineNumber.

    IF the line is empty THEN

      CONTINUE.

    END IF.

    PARSE the CSV record.

    IF the record does not contain

      the expected number of fields THEN

      RECORD "Damaged record" with LineNumber.

      CONTINUE.

    END IF.

    VALIDATE field formats and checksum.

IF validation fails THEN

RECORD "Invalid record" with LineNumber.

CONTINUE.

END IF.

IF record Pass ID equals required Pass ID THEN

DISPLAY the matching record.

CLOSE the file.

RETURN the record.

END IF.

END WHILE.

6. CLOSE the file.

7. DISPLAY "Pass not found".

END.

Explanation

1. The user enters the Pass ID.
2. The application opens the file using BufferedReader.

3. It reads records one line at a time.
4. Each line is parsed into fields.
5. Damaged records are skipped.
6. Valid records are compared with the required Pass ID.
7. When a match is found, the record is returned.
8. If no matching record is found, a not-found message is displayed.

Stream Justification

FileReader reads character data.

BufferedReader reads complete lines using readLine() and reduces repeated low-level read operations.

4.4 Operation 3 – List All Passes on a Given Route

Aim: To display all bus passes belonging to a particular route number.

Pseudocode

ALGORITHM: LIST_PASSES_BY_ROUTE

INPUT: Required Route Number.

OUTPUT: All valid bus-pass records belonging to the selected route.

BEGIN

1. READ the required Route Number.

2. IF the text file does not exist THEN
    DISPLAY "Bus-pass file not found".
    RETURN.
  END IF.

3. OPEN the file using:
    FileReader.
    BufferedReader.

4. SET Found = FALSE.

5. WHILE file has another line DO

    READ the next line.

    IF the line is empty THEN
        CONTINUE.
    END IF.

    PARSE the CSV record.

    IF the record has an incorrect number of fields THEN
        RECORD the damaged line.
        CONTINUE.
    END IF.

    VALIDATE the field formats and checksum.

    IF validation fails THEN
        RECORD the damaged line.
        CONTINUE.
    END IF.

    EXTRACT Route Number.

    IF Route Number equals required Route Number THEN

        DISPLAY the bus-pass record.

SET Found = TRUE.

END IF.

END WHILE.

6. CLOSE the file.

7. IF Found = FALSE THEN

DISPLAY "No passes found for this route".

END IF.

END.

Explanation

1. The user enters the route number.
2. The application opens the file using BufferedReader.
3. It reads every record.
4. It checks the number of fields and validates the values.
5. Damaged records are skipped.
6. Valid records belonging to the requested route are displayed.
7. If no matching records are found, an appropriate message is shown.

Stream Justification

BufferedReader is suitable because the operation must examine all records one line at a time.

The application does not need to load the entire file into memory.

4.5 Summary of Stream Selection

Operation	**Stream Classes**	**Reason**
Add pass	FileWriter, BufferedWriter	Appends text records efficiently

Find pass	FileReader, BufferedReader	Reads and searches line by line
List by route	FileReader, BufferedReader	Examines each record efficiently
Binary writing	FileOutputStream, DataOutputStream	Writes primitive values and strings in binary form
Binary reading	FileInputStream, DataInputStream	Reads values in the same binary order

5. Task A1.3 – Design for Damaged Data (2 Marks)

A file may become damaged because of incorrect input, unexpected shutdowns, storage problems, or incomplete writing.

The design must handle three major situations.

5.1 Damaged Line

A damaged line may contain:

- Fewer fields than expected.
- More fields than expected.
- Missing Pass ID.
- Invalid route number.
- Incorrect validity date.
- Invalid amount.
- Incorrect checksum.
- Incorrect CSV quoting.

Example of a damaged record:

P1002,192311002

This record contains only two fields instead of the required six business fields.

Another example:

P1003,192311003,Ravi Kumar,ABC,2027-12-31,4500

The route number is invalid because it contains letters.

Handling Method

1. Read the record line by line.
2. Parse the CSV fields.

3. Check the expected number of fields.
4. Validate each field's data type and format.
5. Verify the checksum.
6. If the record is damaged, skip it.
7. Record the line number and error reason in a separate error log.
8. Continue processing the remaining records.

Example Output:

Line 2: Damaged record - incorrect number of fields.

Line 3: Invalid record - route number must be numeric.

Valid records processed successfully.

The entire file should not stop processing because of one damaged record.

5.2 Missing File

The file may not exist because it has not yet been created, was deleted, or was moved.

Handling Method

For an Add operation:

- Use FileWriter in append mode.
- If the file does not exist, CREATE it.
- Write the new record.

For Find and List operations:

- Check whether the file exists before reading.
- If it does not exist, display a clear message.
- Do not terminate the application unexpectedly.

Example Output:

Error: Bus-pass file not found.

The application should distinguish between a missing file and an empty file.

5.3 Incomplete Write Caused by Power Failure

A power failure may occur while a new record is being written.

For example, a record may be partially stored:

P1004,192311004,Arun Kumar,12,2027-

The record is incomplete and cannot be treated as valid.

Handling Method

The following techniques are used:

1. Newline Verification

A complete record should end with a record separator.

An incomplete final line can be identified during recovery.

2. Checksum Verification

A checksum is stored with each record.

If the stored record has been partially written or altered, checksum verification can detect the corruption.

3. Record Validation

The application checks that every field is present and has the correct format.

4. Temporary File

For operations that rewrite the complete file, first write the updated content into a temporary file.

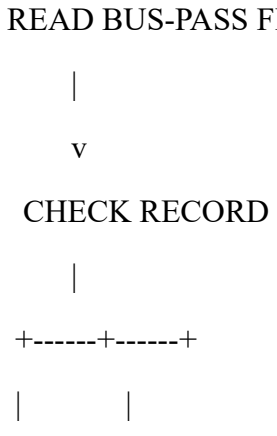
After successful completion, replace the original file using an atomic move where the file system supports it.

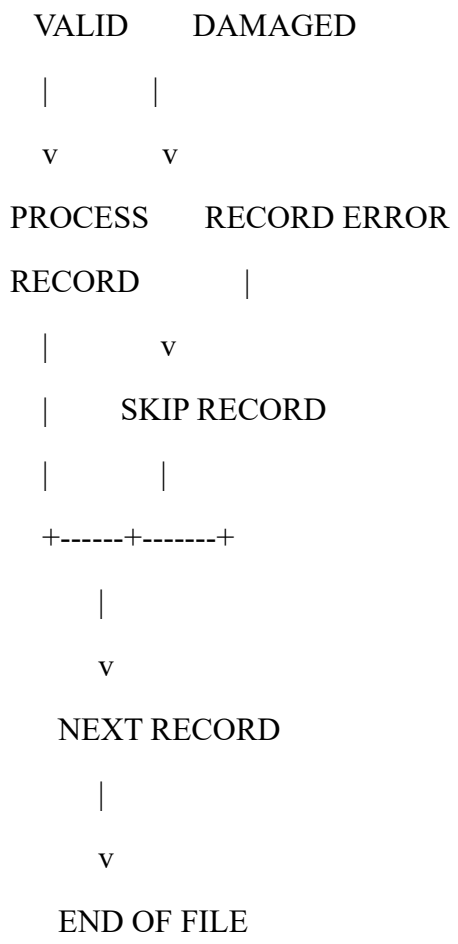
5. Recovery Log

Maintain a backup or journal of the last confirmed operation so that incomplete updates can be identified and recovered.

A buffered writer's flush() alone does not guarantee that data has reached permanent storage. For stronger durability, a file channel can be forced to storage.

Error Recovery Diagram





6. Alternative Concept and Complexity Analysis

6.1 Alternative – RandomAccessFile with Fixed-Length Records

Instead of storing variable-length text records, the application can use Java's RandomAccessFile.

RandomAccessFile supports reading and writing at particular file positions.

Each record is assigned a fixed size.

Therefore, the address of record i can be calculated using:

Offset = i × RecordSize

The application can use seek() to move directly to that position.

6.2 Fixed-Length Record Layout

The following fixed-size layout is selected.

Field	**Fixed Size**
Pass ID	12 bytes

Register Number	15 bytes
Student Name	60 bytes
Route Number	4 bytes
Validity Date	8 bytes
Amount Paid	8 bytes
Total Record Size	**107 bytes**

String fields are stored using fixed-width UTF-8 byte fields with padding.

The numeric fields use fixed-width binary representations.

The application must reject values that exceed the maximum field capacity instead of silently truncating them.

6.3 Example of Seeking a Record

Assume each record occupies 107 bytes.

To access record number 5 using zero-based indexing:

Offset = 5 × 107

Offset = 535 bytes

The application can use:

```
RandomAccessFile file =
    new RandomAccessFile("buspasses.dat", "rw");

file.seek(535);
```

After seeking, the application reads the fields in the predefined order.

6.4 Finding a Pass by ID

There are two possible approaches.

Approach 1 – Sequential Search

The application reads each record and compares its Pass ID with the requested ID.

Time complexity:

O(n)

Here, n represents the total number of records.

Approach 2 – Direct Access Using an Index

Maintain an index that stores:

Pass ID → Record Number

Example:

P1001 → 0

P1002 → 1

P1003 → 2

P1004 → 3

The application first searches the index to obtain the record number.

It then calculates the file offset:

Offset = Record Number × Record Size

Using a HashMap, the average lookup time is O(1), and the file seek is also O(1).

Therefore, the average search time becomes:

O(1)

This direct-access result assumes the Pass ID has a maintained index. RandomAccessFile alone does not make an arbitrary ID search O(1); without an index, it still needs an O(n) scan.

6.5 Time Complexity Comparison

Operation	**Delimited Text File**	**RandomAccessFile**
Add pass	O(1) append	O(1) append
Find by ID without index	O(n)	O(n)
Find by ID with index	O(n) scan	O(1) average lookup and seek
List by route	O(n)	O(n)
Access by known record number	O(n) sequential access	O(1) seek

6.6 Space Wasted by Padding

Fixed-length records reserve the maximum possible size for every field.

Consider the selected record layout:

Pass ID = 12 bytes

Register Number = 15 bytes

Name = 60 bytes

Route Number = 4 bytes

Validity Date = 8 bytes

Amount Paid = 8 bytes

Total = 107 bytes

Suppose the actual data requires:

Pass ID = 6 bytes

Register Number = 9 bytes

Name = 10 bytes

Route Number = 4 bytes

Validity Date = 8 bytes

Amount Paid = 8 bytes

Actual total = 45 bytes

Unused space:

107 - 45 = 62 bytes

Therefore, 62 bytes are wasted for this example record.

For 8,000 similar records:

62 × 8000 = 496000 bytes

Approximately 484.4 KiB would be unused due to padding.

Actual waste depends on the lengths of the stored names and identifiers.

6.7 Comparison of Both Storage Designs

Feature	**Delimited Text File**	**RandomAccessFile**
Record size	Variable	Fixed
Human readability	Yes	No

Search without index	O(n)	O(n)
Search with index	Usually O(n) without building an index	O(1) average lookup and seek
Storage efficiency	Depends on text length	Padding may waste space
Direct record access	Difficult	Supported
Record modification	Usually requires rewriting	Supports direct updates
Complexity	Simple	Requires fixed layout and indexing

6.8 Engineering Decision

For approximately 8,000 bus-pass records per year, a delimited text file is a suitable initial design.

It is easy to read, modify, back up, and transfer.

However, if the transport office requires frequent direct searches and updates, RandomAccessFile with a maintained index can provide faster access.

Fixed-length records also create padding overhead and require strict limits on string-field sizes.

7. Final Result

Two file formats were designed for the university bus-pass record system: a delimited CSV text file and a binary file using DataOutputStream.

Algorithmic pseudocode was developed for adding a pass, finding a pass by Pass ID, and listing passes by route number.

The design uses buffered character streams for text files and byte streams for binary files.

Damaged records, missing files, and incomplete writes caused by power failures were analysed with suitable detection and recovery methods.

Finally, the delimited text-file approach was compared with RandomAccessFile using fixed-length records, including search complexity and wasted storage due to padding.

Thus, the file-storage design provides a structured and reliable approach for managing university bus-pass records.